
CAGT

Recall AI
Software Architecture Document

Version 2.0

Recall AI	Version: 2.0
Software Architecture Document	Date: 14/Aug/26
RecallAI-SAD-v2.0	

Revision History

Date	Version	Description	Author
26/Jul/26	1.0	Initial SAD for PA3: Introduction, Architectural Goals and Constraints, Use-Case Model, and Logical View (Architectural Representation, Use-Case-to-Component Mapping, AI Agents, and the Authentication / Study-Plan and Concept-Graph / Database components with class and ER diagrams). Deployment View and Implementation View left blank per template guidance (PA4 deliverable).	Nguyen The Quan
12/Aug/26	1.1	Added the AI Examiner component to the Logical View (interview state machine, C4 boundary, Flashcard fallback) + state-diagram figure; SRE traceback/scheduling recorded as implemented in Sprint 4	Nguyen The Quan
14/Aug/26	2.0	Filled §5 Deployment View and §6 Implementation View (PA4 deliverable); added §4.7 AI Examiner class diagram (Figure 4.6) + Key classes table; renumbered Figures 4.6→4.9 accordingly; refreshed §1 Overview to reflect PA4 scope	Ngo Van Phong

Recall AI	Version: 2.0
Software Architecture Document	Date: 14/Aug/26
RecallAI-SAD-v2.0	

Table of Contents

1. Introduction	4
2. Architectural Goals and Constraints	4
3. Use-Case Model	5
Changes since PA3	9
Admin actor	10
4. Logical View	10
4.1 Architectural Representation	10
4.2 Use-Case-to-Component Mapping	12
4.3 AI Agents	12
4.4 Component: Authentication	12
4.5 Component: Study-Plan & Concept-Graph	14
4.6 Component: Database	15
4.7 Component: AI Examiner	16
Key classes	17
4.8 Component: Focus Session	19
Overview	19
Key classes	21
Design notes	21
4.9 Component: Dashboard & Visualization	21
Overview	21
Key classes	22
Design notes	23
5. Deployment	24
5.1 Overview	24
5.2 Node Descriptions and Protocols	24
5.3 Notes	25
6. Implementation View	25
6.1 src/client - Presentation Tier	25
6.2 src/server - Application Tier	26
6.3 Notes	26

Recall AI	Version: 2.0
Software Architecture Document	Date: 14/Aug/26
RecallAI-SAD-v2.0	

Software Architecture Document

1. Introduction

Purpose: This Software Architecture Document (SAD) describes the software architecture of Recall AI, an AI-assisted study application built for the Introduction to Software Engineering course. It records the architectural decisions made about the system, gives the team a common vocabulary, and serves as the PA4 architecture deliverable.

Scope: This document covers the components implemented as of the current sprint of the Recall AI MVP: user authentication, study-plan creation with AI-assisted concept extraction, the concept-graph data model, the underlying database, the AI Examiner multi-turn interview engine, and the Focus Session component. Dashboard & Visualization's use cases are largely served by these already-documented components (§4.9) rather than owning a dedicated backend service of its own.

Definitions, Acronyms, and Abbreviations: SAD - Software Architecture Document. SRE - Scheduling & Remediation Engine, the system's internal deterministic scheduling/traceback module. DAG - Directed Acyclic Graph. NFR - Non-Functional Requirement. SPA - Single-Page Application. JWT - JSON Web Token. UC - Use Case. MVP - Minimum Viable Product. C4/C5/C6 - the three hard architectural constraints defined in the Software Development Plan (see Section 2).

References: Vision Document v1.0 (PA1); Software Development Plan v1.2 (PA2); Use-Case Specification v2.0 (PA3); schema.prisma (source of truth for the Database component).

Overview: Section 2 lists the architectural goals and constraints. Section 3 references the use-case model. Section 4 (Logical View) presents the 3-tier architectural representation, the use-case-to-component mapping, the AI agents, and the design of each implemented component. Sections 5 and 6 (Deployment View, Implementation View), left blank in PA3 per the template's guidance, are completed in this PA4 revision.

2. Architectural Goals and Constraints

The following goals and constraints carry over from the Vision Document and the Software Development Plan (Section 2.2); they shape every design decision described in Section 4.

- Platform: the system is a web-based Single-Page Application (SPA), built with React + TypeScript, and must run in modern desktop browsers (Chrome, Safari, Edge, Firefox).
- External dependency: the AI Study Planner and AI Examiner depend on an active connection to the external Google Gemini API; the system must degrade gracefully (see Reliability below) rather than fail outright when that dependency is unavailable.
- Cost & resource constraint: zero budget - every tool and service runs on a free tier (Gemini API free tier, free hosting, GitHub, Figma). AI usage is hard-capped to control cost and latency: a maximum of 3 question-answer turns per concept per Interview session (constraint C6).
- C4 - AI is never the orchestrator: every routing decision (when to stop a turn, when to switch concepts, when to trigger a concept traceback) is deterministic software logic, never left to the language model. The AI is invoked only for narrow, fixed-JSON-schema tasks.
- C5 - the AI Examiner must never fabricate questions outside the student's uploaded material; no external knowledge is injected into generated content.
- Reliability - AI Fallback Mechanism: if the Gemini API fails, times out, or exceeds its quota, the system degrades to a self-grading mode using previously cached questions (static flashcards) instead of failing the session.
- Data integrity: the system uses a relational database (PostgreSQL, ACID transactions) so the concept graph can never be left with orphaned concepts or corrupted prerequisite edges.
- Cycle prevention: every concept graph, whether AI-generated or student-edited, is validated as a Directed Acyclic Graph (DAG) before being saved; cyclic edges are rejected or automatically removed and logged, never silently accepted.
- Schedule: a fixed 10-week, 5-sprint schedule with no possibility of extension, and a team fixed at 5 members - architecture choices favor implementation speed and free-tier services over long-term scalability.

Recall AI	Version: 2.0
Software Architecture Document	Date: 14/Aug/26
RecallAI-SAD-v2.0	

3. Use-Case Model

Recall AI's use-case model is organized into five modules: Account Management (AM), AI Study Planner (SP), Focus Session (FS), AI Examiner (AE), and Dashboard & Visualization (DB), totalling 42 use cases across all planned sprints. The diagrams below, already produced for the use-case model (PA2) and still structurally valid after the Use-Case Specification v2.0 revision (PA3), show the actors and use cases per module. Full flows, pre/post-conditions, and the three use cases added in v2.0 (Dashboard Overview, Study Material Management, Session History) are detailed in the Use-Case Specification v2.0 document.

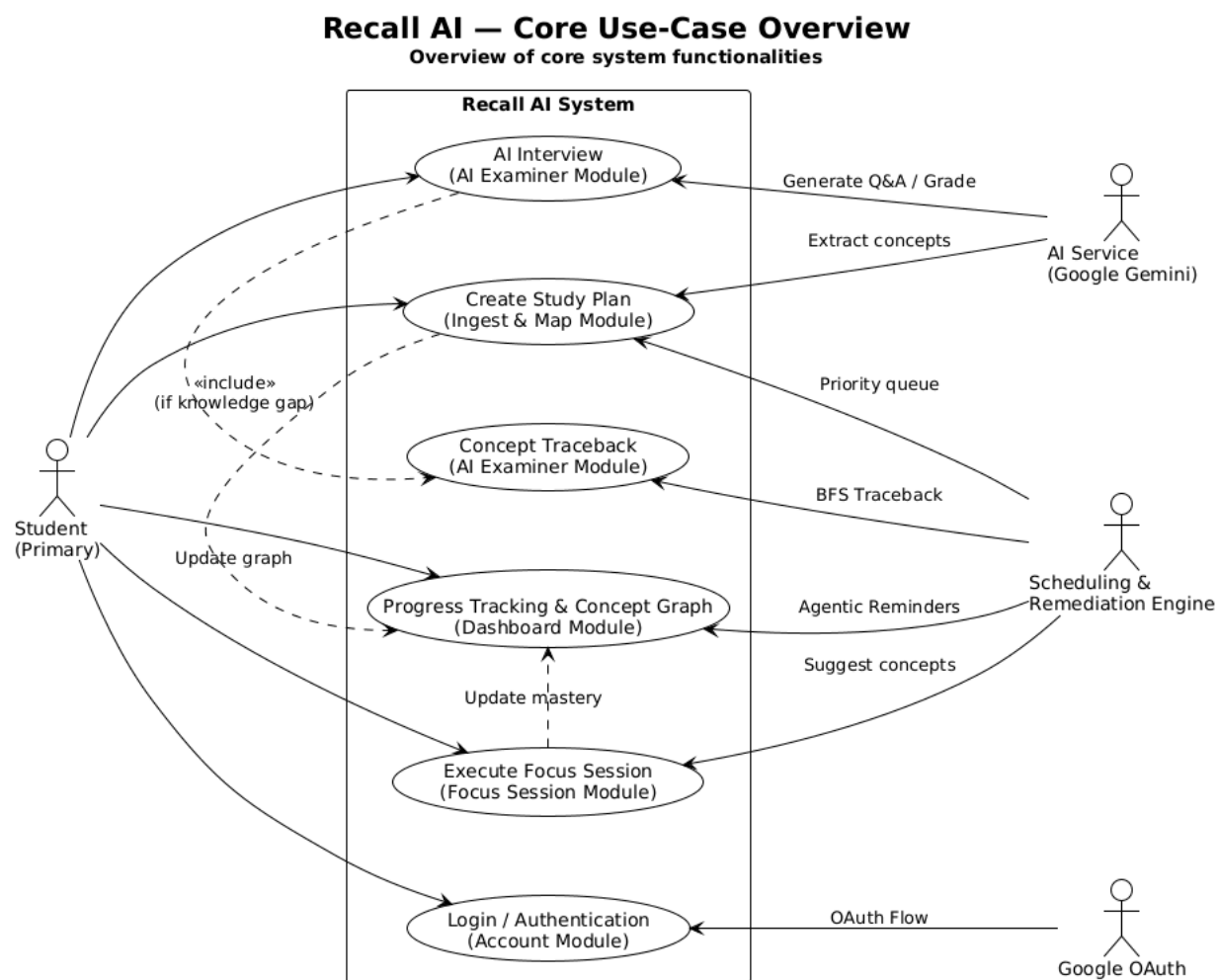


Figure 3.1 - Use-Case Overview

Recall AI	Version: 2.0
Software Architecture Document	Date: 14/Aug/26
RecallAI-SAD-v2.0	

MODULE 1: Account Management

Recall AI — Use-Case Diagram

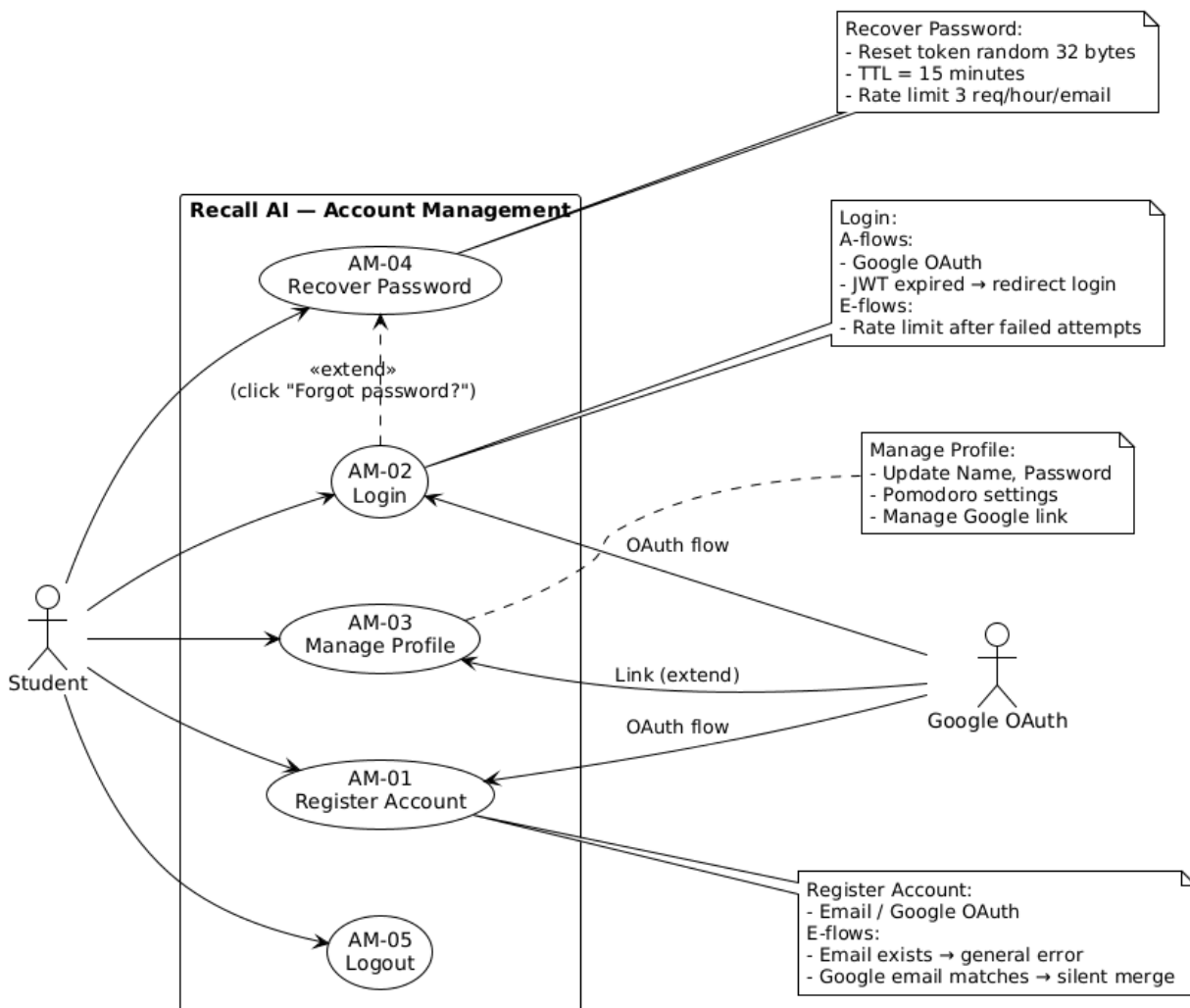


Figure 3.2 - Account Management (AM)

Recall AI	Version: 2.0
Software Architecture Document	Date: 14/Aug/26
RecallAI-SAD-v2.0	

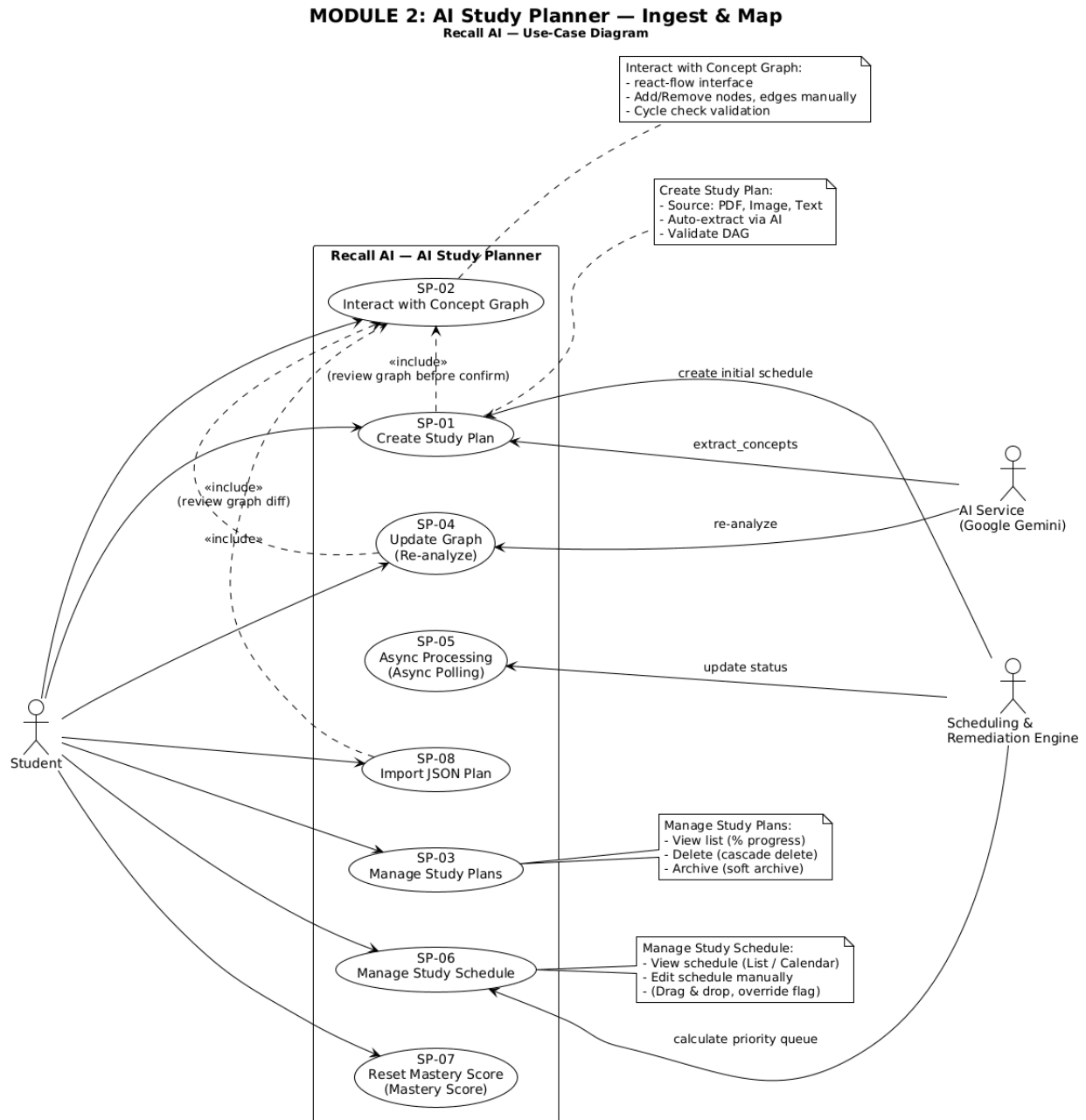


Figure 3.3 - AI Study Planner (SP)

Recall AI	Version: 2.0
Software Architecture Document	Date: 14/Aug/26
RecallAI-SAD-v2.0	

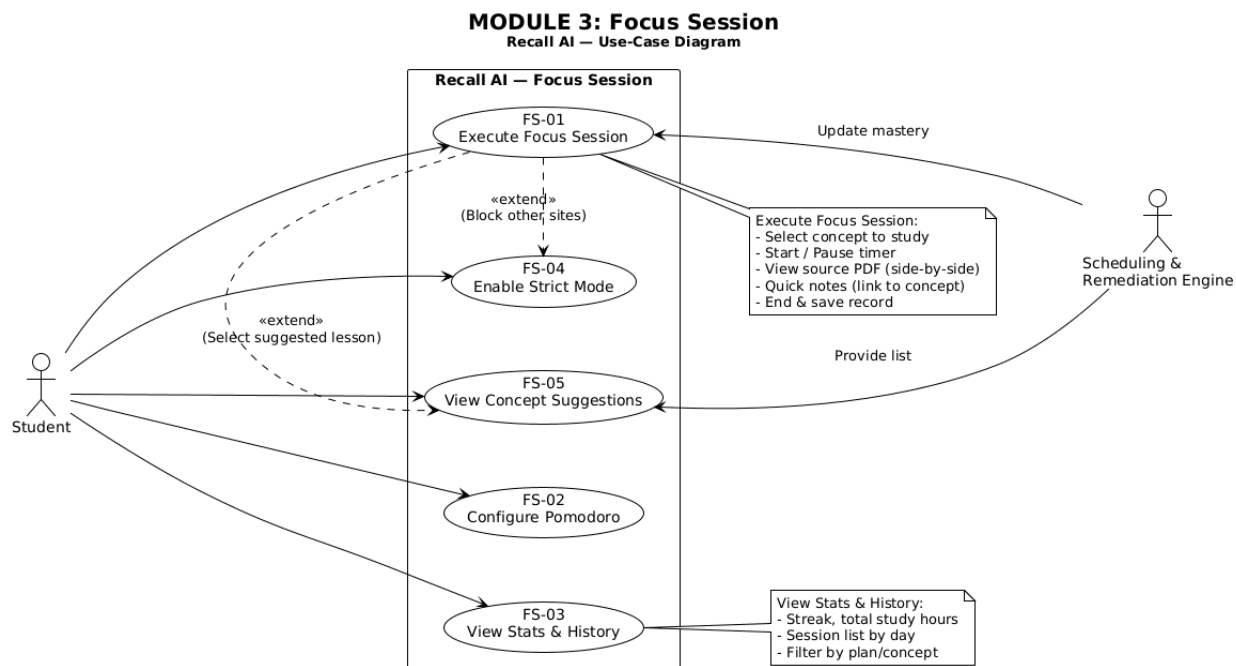


Figure 3.4 - Focus Session (FS)

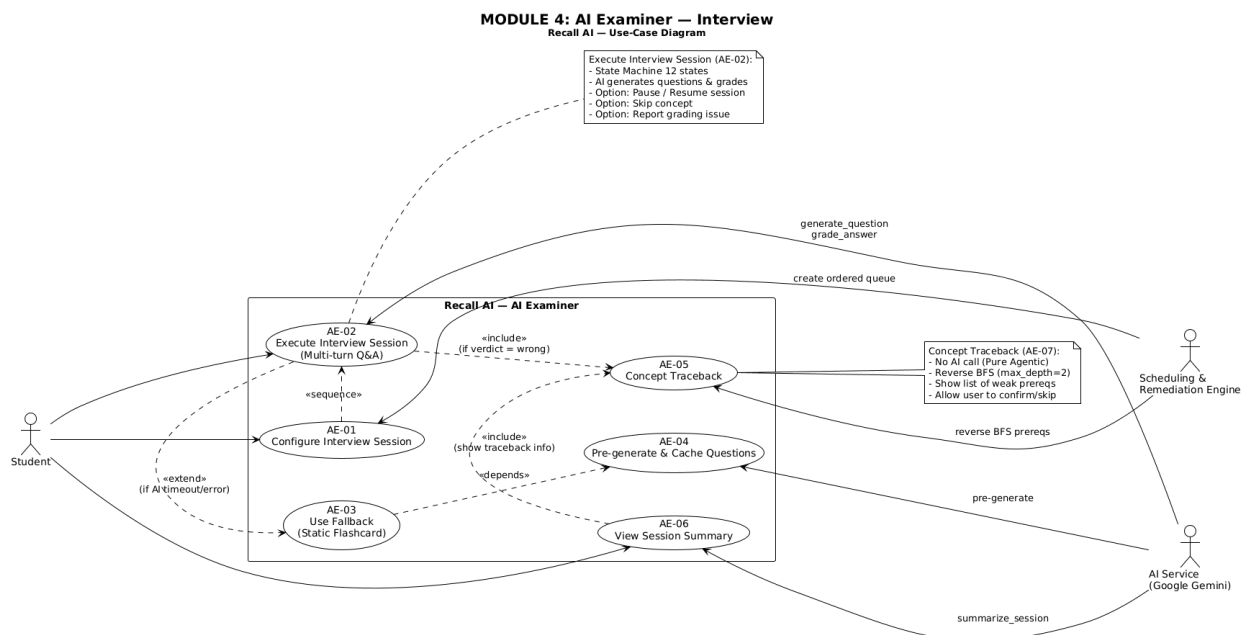


Figure 3.5 - AI Examiner (AE)

Recall AI	Version: 2.0
Software Architecture Document	Date: 14/Aug/26
RecallAI-SAD-v2.0	

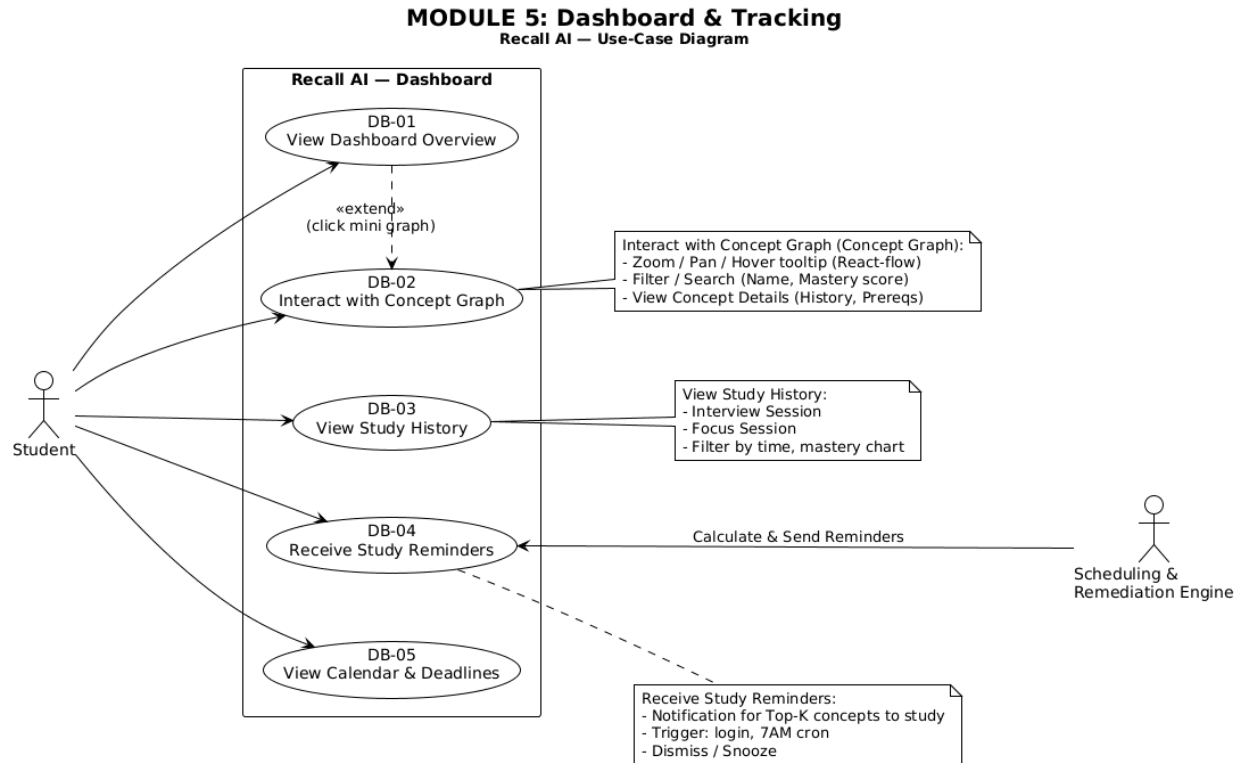


Figure 3.6 - Dashboard & Visualization (DB)

Changes since PA3

The use-case set is unchanged since PA3 (Use-Case Specification v2.0) - still 42 use cases across 5 modules (AM, SP, FS, AE, DB); no use case has been added or removed. What changed between the PA3 submission and this revision:

CHANGE	DETAIL
New actor: Admin (documentation-only)	Added per decision 05/08/2026 (#249). No UC, table, endpoint, or screen - see the actor note below.
Sprint-label corrections (chốt 2026-08-11)	FS-04, FS-05: relabelled Sprint 4-5 → Sprint 4 (#227, #228, closed). DB-09: Sprint 5 → Sprint 4 (#233, closed).
DB-07 (Deadline Calendar) moved to POST-MVP	Decision #234 removed the dead "Xem lịch →" link instead of building the screen; no issue exists for it, so it now sits outside MVP scope.
generate_question AI schema fixed (chốt 2026-08-11)	Response shape dropped the redundant concept_id field. UC text unchanged, only the underlying schema.
New table session_notes landed for FS-05	Ghi chú nhanh trong phiên học now has its 4 endpoints and the session_notes table (§4.6 Database ER redraw).

Recall AI	Version: 2.0
Software Architecture Document	Date: 14/Aug/26
RecallAI-SAD-v2.0	

The three use cases added at PA3 itself (Dashboard Overview, Study Material Management, Session History - detailed in Use-Case Specification v2.0) remain unchanged in this revision.

Admin actor

An Admin actor has also been added to the system's role model at a documentation-only level (decision 05/08/2026, #249). Recall AI's current schema and use-case diagrams reflect Student as the only account type - the User model in `schema.prisma` has no `role` column, and no administrative use case, endpoint, or screen is implemented in this MVP. Admin is recorded here purely to complete the actor set for future planning; when it is actually implemented, it will need to be attached to concrete administrative use cases and reflected in the diagrams above.

4. Logical View

4.1 Architectural Representation

Recall AI follows a 3-tier architecture: a Presentation tier (the React SPA), an Application tier (the Express.js REST API, including the internal Scheduling & Remediation Engine), and a Data tier (PostgreSQL via Prisma). The Application tier additionally depends on one external service, the Google Gemini API, for AI-assisted extraction and grading tasks.

Recall AI	Version: 2.0
Software Architecture Document	Date: 14/Aug/26
RecallAI-SAD-v2.0	

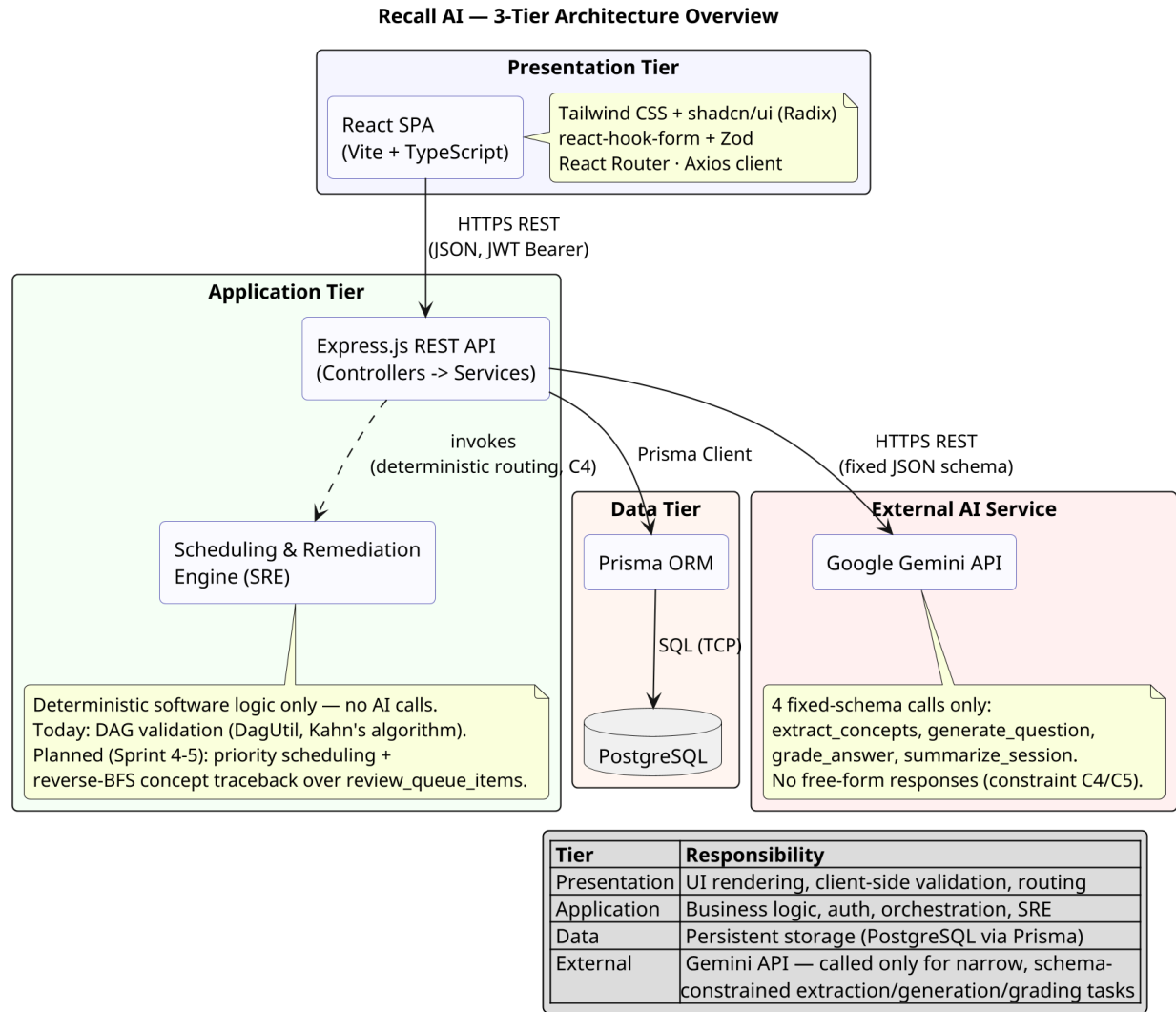


Figure 4.1 - 3-Tier Architecture Overview

The table below lists the technology used in each tier.

Tier	Technology
Presentation	React 19 (Vite) + TypeScript; Tailwind CSS v4 + shadcn/ui (Radix primitives); react-hook-form + Zod; React Router v7; Axios HTTP client
Application	Node.js + Express.js 5 (TypeScript); Zod request validation; JWT (jsonwebtoken) + bcryptjs; Multer file upload; Helmet + CORS
Data	PostgreSQL, accessed through Prisma ORM 7
External AI Service	Google Gemini API, via the official @google/genai SDK

Components communicate over HTTPS using REST endpoints under /api/v1, exchanging JSON payloads. The Presentation tier authenticates with a JWT Bearer token issued by the Authentication component. The Application

Recall AI	Version: 2.0
Software Architecture Document	Date: 14/Aug/26
RecallAI-SAD-v2.0	

tier talks to the Data tier through the Prisma Client (SQL over a TCP connection) and to the external AI service over HTTPS REST with a fixed, pre-declared JSON response schema - never a free-form response.

4.2 Use-Case-to-Component Mapping

The table below maps each use-case module to the component currently responsible for it.

Use-Case Module	Responsible Component
AM-01 .. AM-05 (Account Management)	Authentication
SP-01, SP-02, SP-05, SP-06 (Create / Edit / Re-analyze Study Plan)	Study-Plan & Concept-Graph
All persisted data for every module (Users, Study Plans, Concepts, Concept Edges, Interview Sessions, Focus Sessions, Review Queue, etc.)	Database
AE-01..AE-10 (AI Examiner)	AI Examiner (§4.7)
FS-01..FS-07 (Focus Session)	Focus Session (§4.8)
DB-01..DB-09 (Dashboard & Visualization)	Dashboard & Visualization (§4.9) - composes Study-Plan & Concept-Graph (§4.5), AI Examiner (§4.7), Focus Session (§4.8), and Scheduling & Remediation Engine rather than owning a dedicated backend service

4.3 AI Agents

Recall AI has two AI/agentic actors, kept strictly separate per constraint C4:

Google Gemini API (external): an external, stateless LLM service invoked only for four fixed-JSON-schema calls - `extract_concepts`, `generate_question`, `grade_answer`, and `summarize_session` (see `gemini.service.ts`). It never decides control flow: every call returns structured JSON validated against a Zod schema before use, and a malformed or free-form response is treated as an error, not as an instruction to follow (constraints C4, C5).

Scheduling & Remediation Engine - SRE (internal): the system's own deterministic engine - no AI calls, fully unit-testable. Today it is implemented as `DagUtil.validateAndFixDag`, which validates and repairs the concept graph using Kahn's algorithm. Its planned Sprint 4-5 responsibilities - priority-queue scheduling and the depth-limited reverse-BFS concept traceback - operate on the `review_queue_items` table already defined in the database schema. Components communicate with both agents over REST APIs using a fixed, pre-declared JSON schema in both directions - the AI Study Planner sends a schema-constrained prompt to Gemini and validates the schema-constrained JSON it gets back; no component ever accepts or forwards a free-form natural-language response as if it were structured data.

4.4 Component: Authentication

The Authentication component implements Account Management (AM-01, AM-02, AM-04) and spans both the Application tier (server) and the Presentation tier (client).

Server: On the server, `AuthController` exposes four HTTP endpoints (`register`, `login`, `refresh`, `me`) and delegates all business logic to `AuthService`, which hashes passwords with `bcrypt` (10 salt rounds), issues a short-lived access token (15 minutes) and a longer-lived refresh token (7 days) via `JwtUtil`, and persists users through the Prisma Client. `AuthMiddleware` guards every protected route by verifying the Bearer token and attaching the authenticated user's id to the request. All errors are raised as the custom `AppError` class (extends `Error`) and handled centrally by `errorHandler`.

Note: `AuthService`, `AuthController`, `AuthMiddleware`, and `JwtUtil` are implemented as stateless functional modules rather than instantiated classes; `AppError` is the only class in this component in the strict OOP sense. The class

Recall AI	Version: 2.0
Software Architecture Document	Date: 14/Aug/26
RecallAI-SAD-v2.0	

diagram below reflects this honestly, using the <<module>>/<<service>> stereotype for the functional modules.

Recall AI — Class Diagram: Authentication Component (back end)

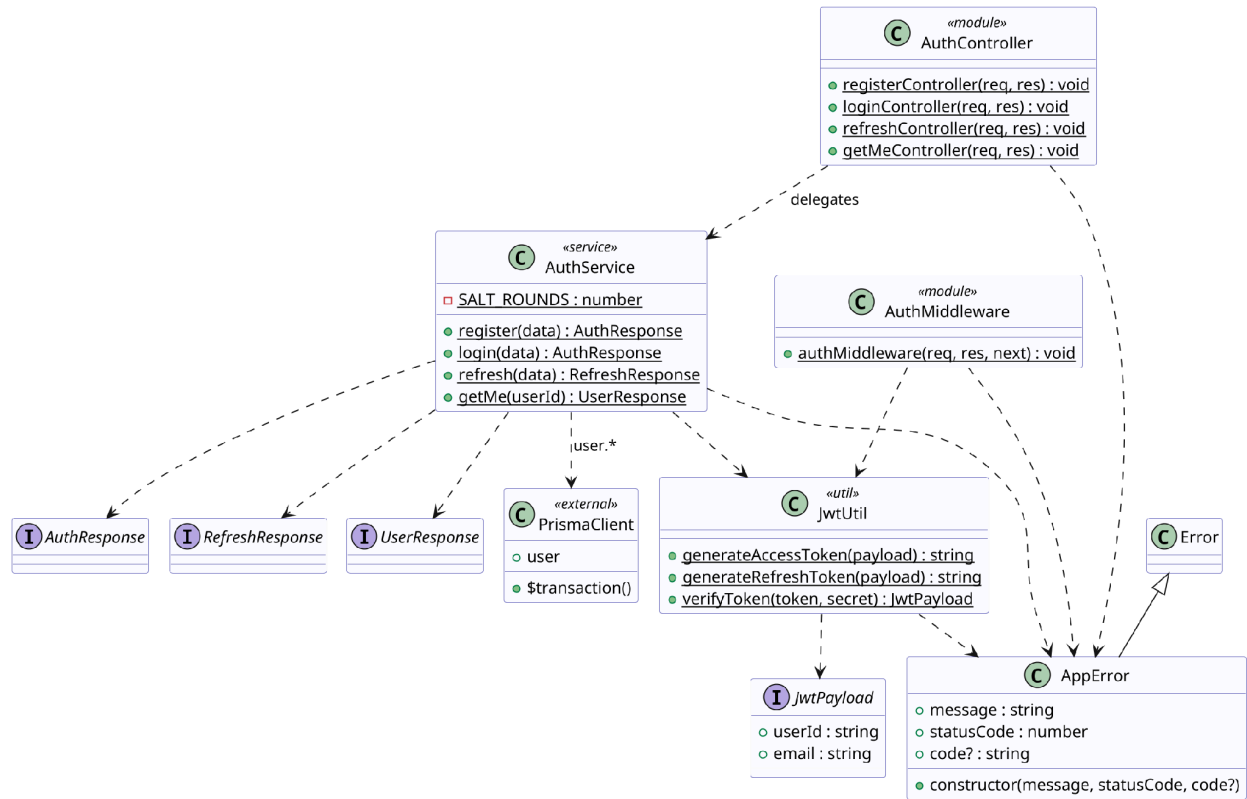


Figure 4.2 - Class Diagram: Authentication Component (Server)

Client: On the client, an AuthProvider React component owns the authentication state and exposes it through a custom useAuth hook backed by AuthContext. LoginForm and SignupForm (react-hook-form + Zod validation) call useAuth().login / .register, and a ProtectedRoute component guards authenticated routes, redirecting to /login when unauthenticated. AuthApi wraps the underlying REST calls, and a single Axios apiClient instance transparently refreshes an expired access token on a 401 response before retrying the original request.

Recall AI	Version: 2.0
Software Architecture Document	Date: 14/Aug/26
RecallAI-SAD-v2.0	

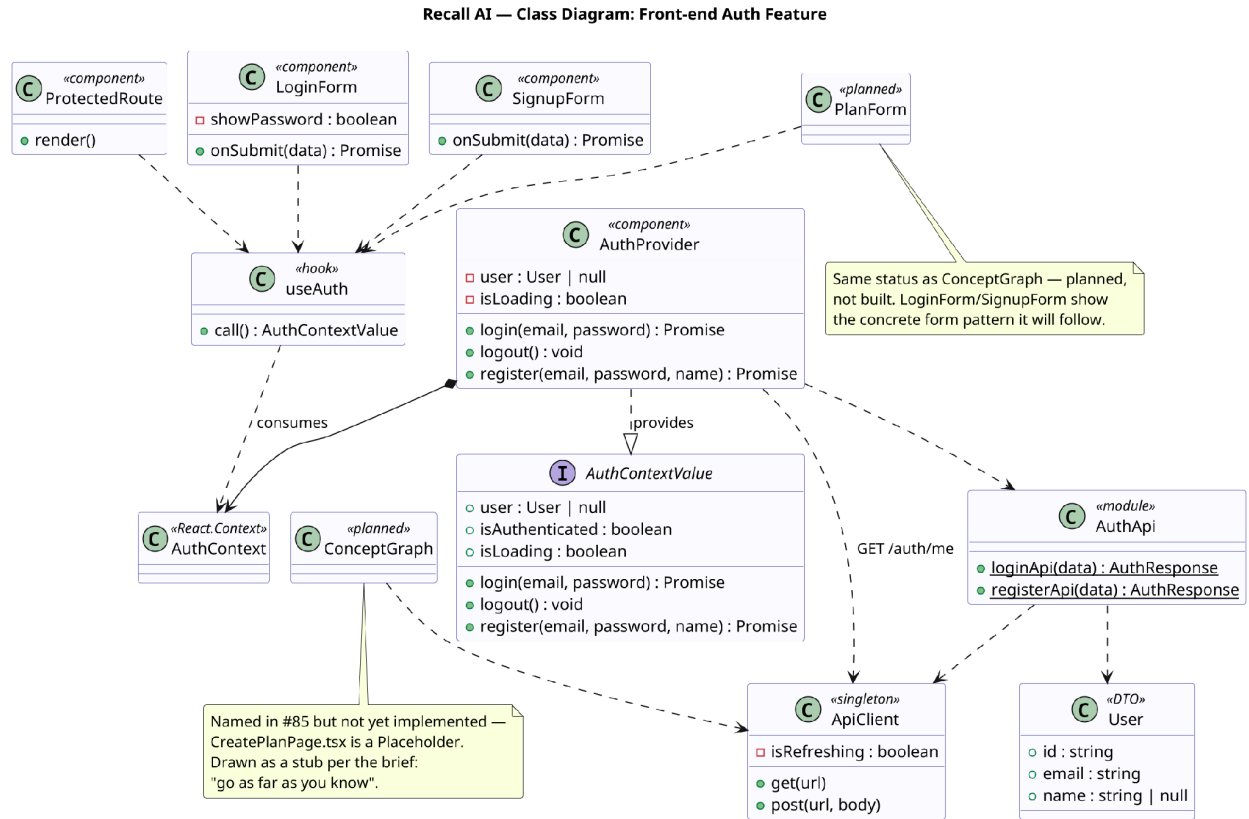


Figure 4.3 - Class Diagram: Authentication Component (Client)

4.5 Component: Study-Plan & Concept-Graph

The Study-Plan & Concept-Graph component implements study-plan creation (SP-01), graph editing (SP-02), and asynchronous document analysis (SP-06). PlanController accepts an uploaded document, stores it through the StorageService abstraction (LocalStorageService for the MVP, created by the createStorageService factory), persists the plan via PlanService, and triggers AnalysisService in the background. AnalysisService calls GeminiService - the only class permitted to talk to the Gemini API - to extract a concept graph, then validates it as a Directed Acyclic Graph using the pure-function DagUtil.validateAndFixDag (Kahn's algorithm) before persisting Concepts and ConceptEdges. GraphService exposes the same DAG validation for the student-facing graph editor (SP-02), rejecting any student edit that would introduce a cycle rather than auto-fixing it.

Recall AI	Version: 2.0
Software Architecture Document	Date: 14/Aug/26
RecallAI-SAD-v2.0	

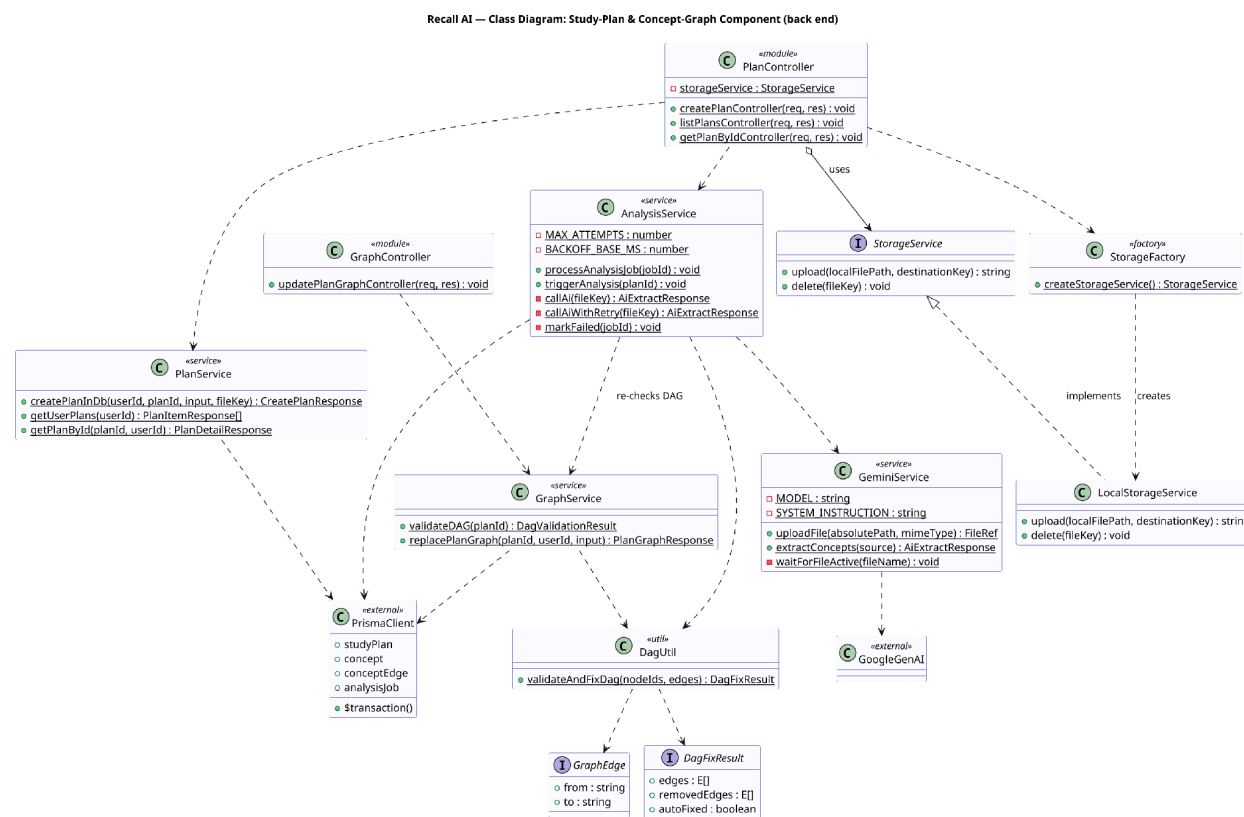


Figure 4.4 - Class Diagram: Study-Plan & Concept-Graph Component

4.6 Component: Database

The Database component persists all application state in PostgreSQL, accessed through the Prisma ORM. The schema currently defines 13 tables and 13 enumerated types: users, study_plans, concepts, and concept_edges (the core concept-graph model, where concept_edges is the associative entity resolving the many-to-many prerequisite relation between concepts); analysis_jobs and question_cache (supporting the Study-Plan component); interview_sessions, interview_turns, focus_sessions, and review_queue_items (the AI Examiner, Focus Session, and Scheduling & Remediation Engine - §4.7, §4.8); and documents, concept_sources, and session_notes (source-document grounding for AI Examiner citations and the Focus Session excerpt view - §4.8, constraint C5). Every foreign key cascades on delete, since the data forms a strict ownership tree rooted at users. A small number of references (for example review_queue_items.source_session_id) are deliberately left as soft references without a foreign-key constraint, where the referenced row's lifecycle is independent of the referencing row. See the companion Database Design document for the full entity-relationship catalogue.

Recall AI	Version: 2.0
Software Architecture Document	Date: 14/Aug/26
RecallAI-SAD-v2.0	

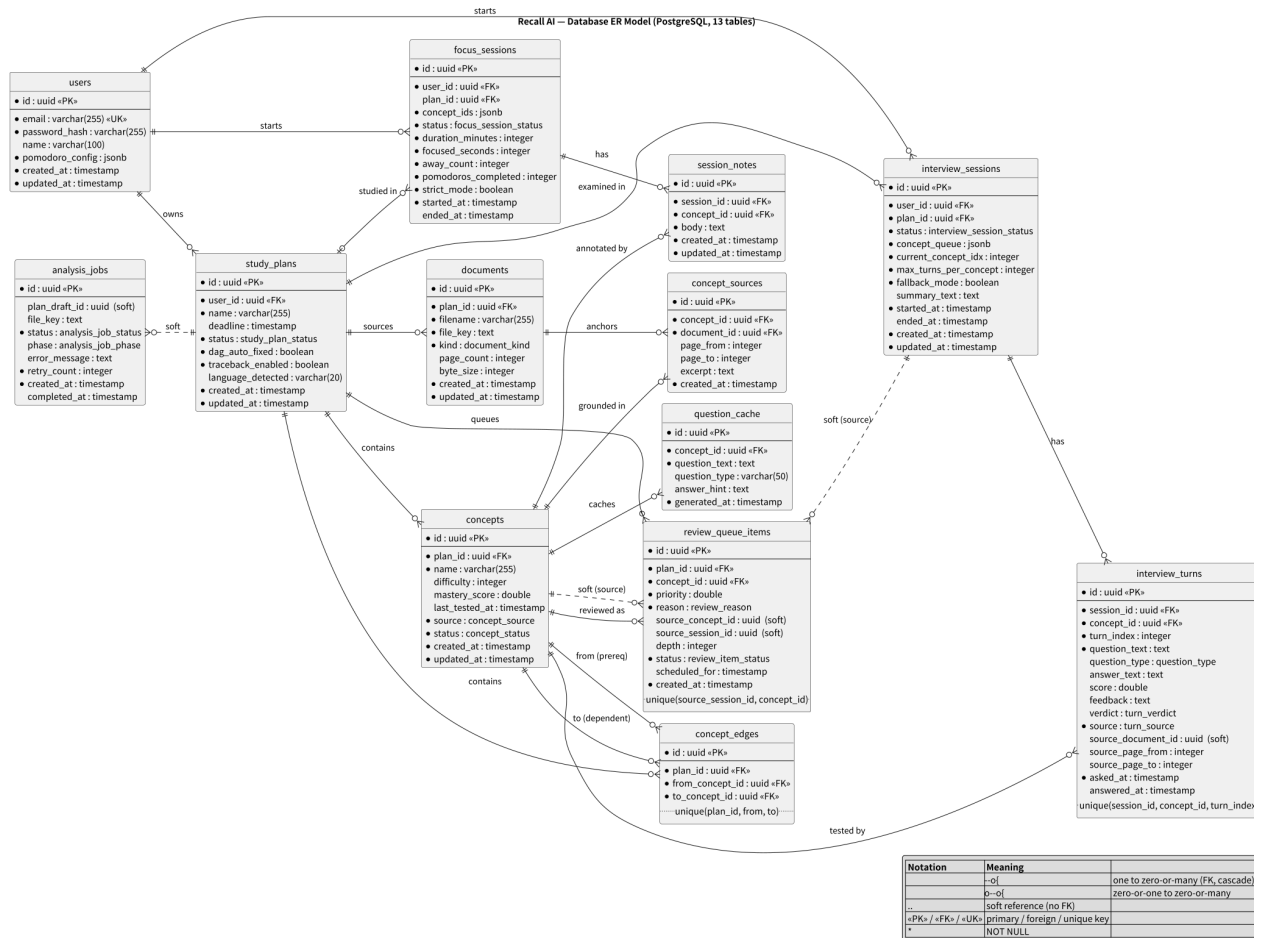


Figure 4.5 - Entity-Relationship Model: Database Component

4.7 Component: AI Examiner

The AI Examiner component implements the multi-turn Interview (EPIC #6, use cases AE-01 through AE-10). InterviewController exposes the session lifecycle - create, retrieve, submit-answer, pause, resume, summary, and abandon - and delegates to InterviewService, which owns the interview state machine, while ConceptResultService finalises each concept and the GeminiService described in Section 4.3 performs the four fixed AI calls.

Recall AI	Version: 2.0
Software Architecture Document	Date: 14/Aug/26
RecallAI-SAD-v2.0	

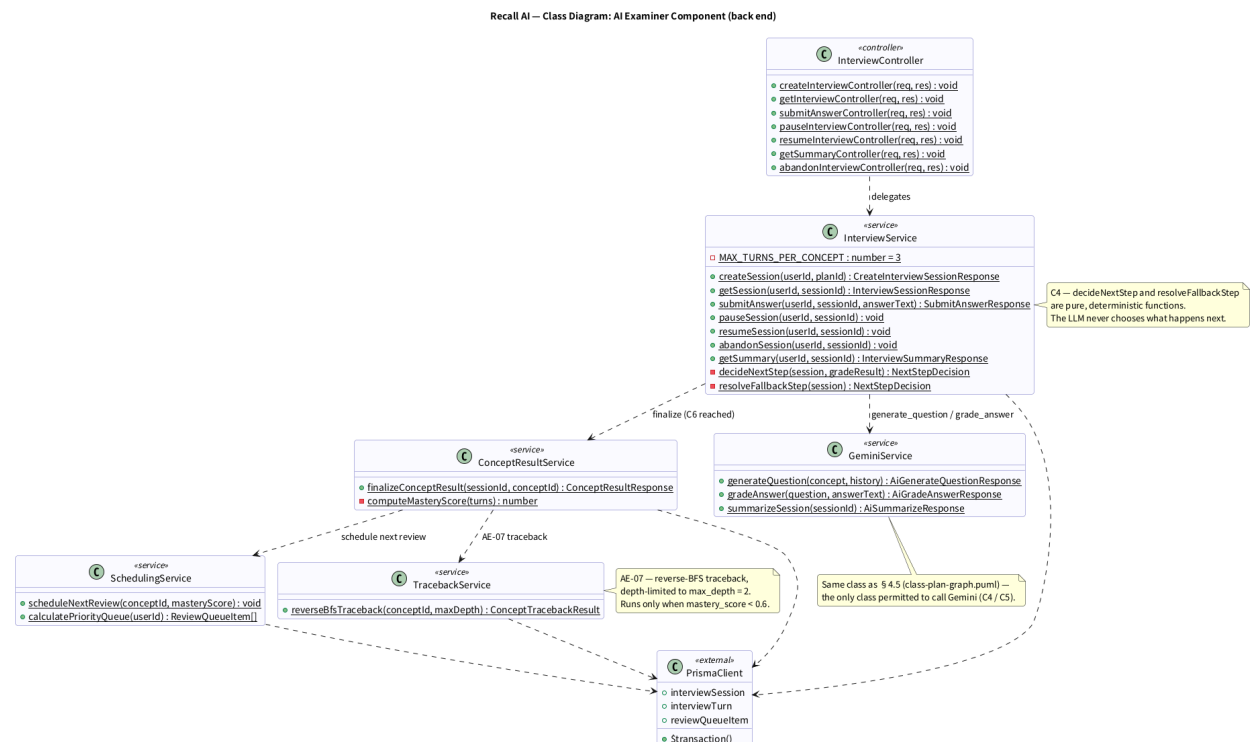


Figure 4.6 – Class Diagram: AI Examiner Component (back end)

Key classes

CLASS	KIND	KEY MEMBERS	RESPONSIBILITY
InterviewController	controller	7 controllers: create, get, submitAnswer, pause, resume, getSummary, abandon	Thin HTTP adapters over InterviewService (AE-01, AE-02, AE-06, AE-08)
InterviewService	service	MAX_TURNS_PER_CONCEPT = 3 (C6); decideNextStep, resolveFallbackStep (private, deterministic)	Owens the interview state machine (Figure 4.7); routes between AI mode and Flashcard fallback - never lets the LLM decide (C4)
ConceptResultService	service	finalizeConceptResult, computeMasteryScore (pure)	Finalises a concept on session end: mastery score, triggers scheduling + traceback
SchedulingService	service	scheduleNextReview, calculatePriorityQueue	Scheduling half of the SRE - writes review_queue_items
TracebackService	service	reverseBfsTraceback (pure, max_depth = 2)	Remediation half of the SRE - AE-07 reverse-BFS traceback, triggered when mastery_score < 0.6
GeminiService	service	generateQuestion, gradeAnswer, summarizeSession	Same class as §4.5 (Figure 4.4) - the only class permitted to call Gemini (C4/C5)

Recall AI	Version: 2.0
Software Architecture Document	Date: 14/Aug/26
RecallAI-SAD-v2.0	

The interview is a deterministic state machine, not an AI-driven dialogue (constraint C4). For each concept in the session queue the examiner asks up to three question-answer turns (constraint C6); after every graded answer the deterministic decideNextStep function - never the language model - decides whether to ask another turn, move to the next concept, or end the session. When a concept ends, ConceptResultService computes a weighted mastery score, schedules the next spaced-repetition review, and runs the depth-limited reverse-BFS prerequisite traceback (AE-07); the Scheduling & Remediation Engine responsibilities that Section 4.3 lists as planned are implemented here. mastery_score and last_tested_at are written only by this component - the Focus Session component updates study statistics but never the mastery score.

The examiner runs two parallel state machines chosen by a single session.fallbackMode flag. On the normal path, questions come from generate_question and grades from grade_answer. Because Gemini is treated as an untrusted, failure-prone dependency, the first failed call in a session flips fallbackMode and the examiner switches - without ending the session - to the Flashcard fallback (AE-05), driven by the deterministic resolveFallbackStep function over pre-generated cached questions that the student self-grades. The reason returned to the client (grading_unavailable, question_unavailable, or no_cached_questions) reports what failed. Cached questions are invalidated whenever the plan's material changes or is re-analysed, so a question generated from replaced material is never served (constraint C5).

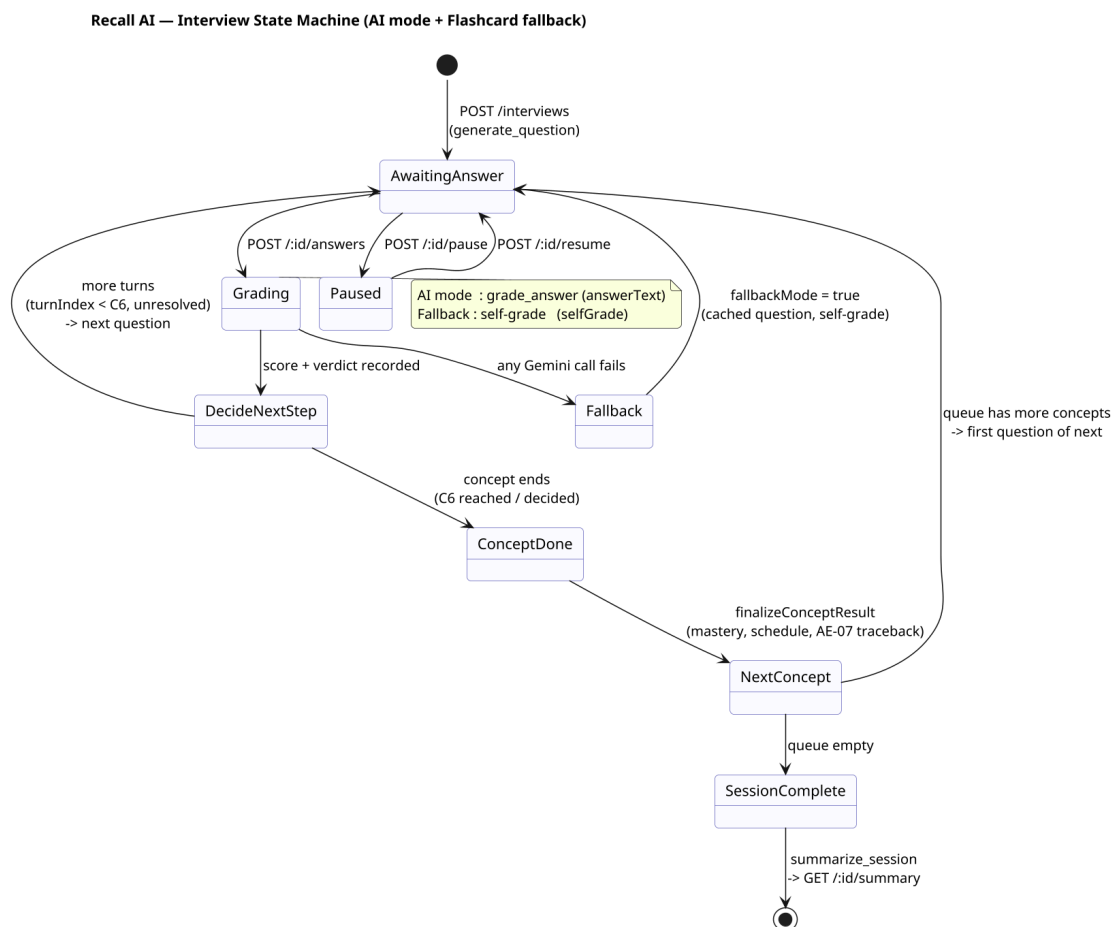


Figure 4.7 – State Diagram: AI Examiner Interview State Machine (AI mode + Flashcard fallback)

Recall AI	Version: 2.0
Software Architecture Document	Date: 14/Aug/26
RecallAI-SAD-v2.0	

4.8 Component: Focus Session

Overview

Focus Session covers FS-01 (Pomodoro study session), FS-03 (session history), and FS-05 (quick notes during a session). Architecturally it is a thin, self-contained CRUD-plus-timer component: the server persists session lifecycle and notes; the Pomodoro clock itself is entirely client-side (no server ticking) so the study timer keeps running through network blips.

Two backend services share one ownership gate: SessionNoteService calls FocusSessionService.getOwnedFocusSessionOrThrow() before touching any note, so the "session belongs to this user" check lives in exactly one place for both the session and its 4 nested note endpoints (/focus-sessions/:id/notes*).

Recall AI	Version: 2.0
Software Architecture Document	Date: 14/Aug/26
RecallAI-SAD-v2.0	

Recall AI — Focus Session Component (back end + key front-end pieces)

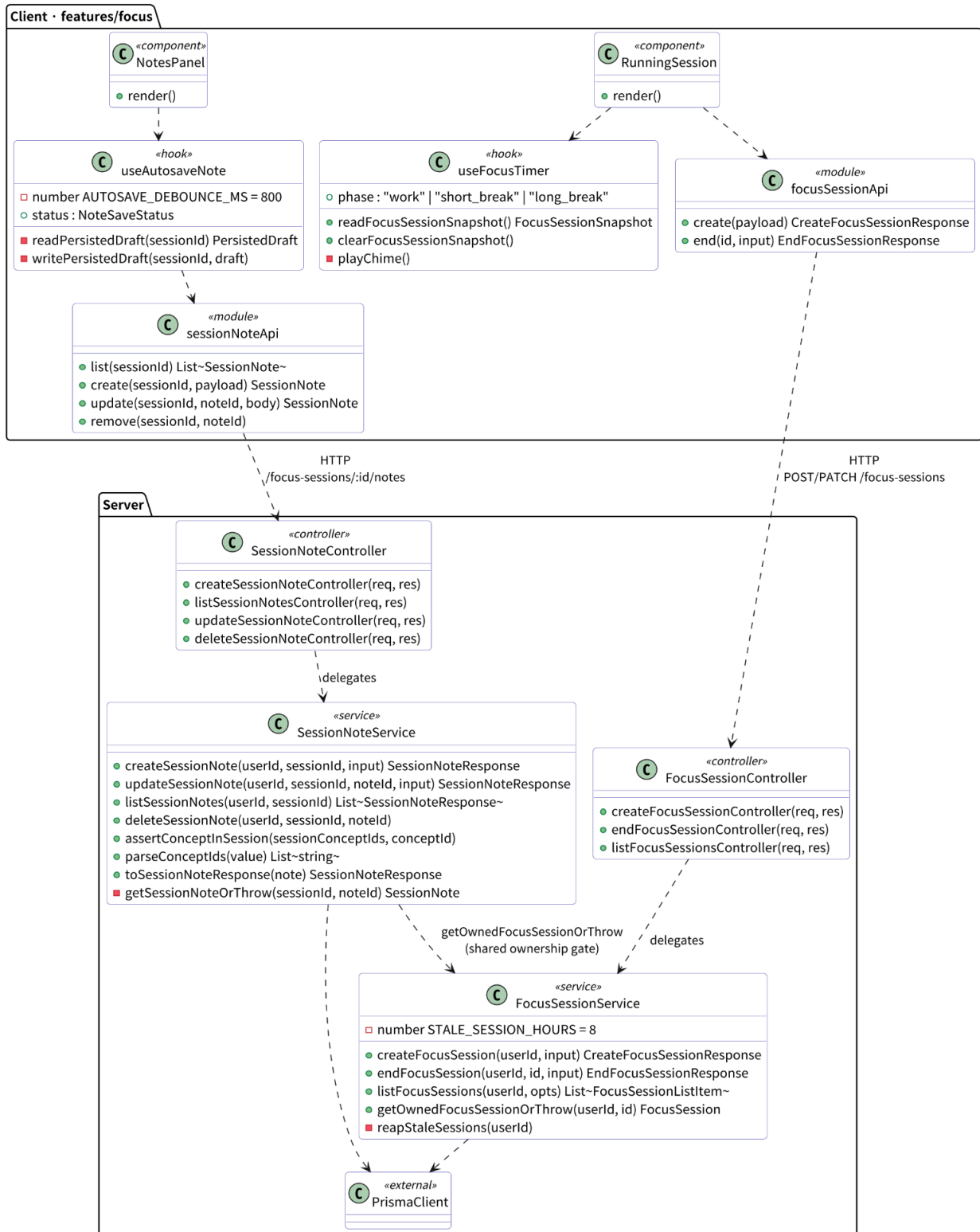


Figure 4.8 – Class Diagram: Focus Session Component (Back End + Key Front-End Pieces)

Recall AI	Version: 2.0
Software Architecture Document	Date: 14/Aug/26
RecallAI-SAD-v2.0	

Key classes

CLASS	KIND	KEY MEMBERS	RESPONSIBILITY
FocusSessionController	module	createFocusSessionController, endFocusSessionController, listFocusSessionsController	Thin HTTP adapters over FocusSessionService
FocusSessionService	module	createFocusSession, endFocusSession, listFocusSessions, getOwnedFocusSessionOrThrow (exported, reused by notes), reapStaleSessions (private, lazy sweep)	Session lifecycle: create (with plan/concept ownership checks), end/cancel, history; never writes mastery_score
SessionNoteController	module	4 controllers, one per note endpoint	HTTP adapters over SessionNoteService, nested under /focus-sessions/:id/notes
SessionNoteService	module	createSessionNote, updateSessionNote, listSessionNotes, deleteSessionNote, assertConceptInSession (pure)	Note CRUD anchored to a concept within the session's own conceptIds
useFocusTimer	hook	phase (work/short_break/long_break), snapshot to localStorage	Client-only Pomodoro engine; resumes after a closed tab
useAutosaveNote	hook	AUTOSAVE_DEBOUNCE_MS = 800, status: NoteSaveStatus	Debounced auto-save; local draft so offline typing isn't lost

Design notes

- Lazy stale-session reaping, no cron. A `running` session older than 8h with no `endedAt` is closed (`status = 'cancelled'`) the next time a user's sessions are read/written.
- Client-owned timer. The Pomodoro clock runs entirely in the browser; the server only sees one `create` and one `end` call per session.
- `mastery_score` boundary shared with §4.7 AI Examiner. Focus Session writes study statistics only; it never touches `concepts.mastery_score` or `last_tested_at`.
- One ownership gate for 5 endpoints. `getOwnedFocusSessionOrThrow` is exported specifically so the notes service reuses the same "not found vs not yours" check (always 404, never 403).

4.9 Component: Dashboard & Visualization

Overview

Dashboard & Visualization (DB-01...DB-09) is architecturally thin on the back end and an aggregation layer on the front end - it is not a monolithic service that owns its own data. The server side is a single read-only endpoint, GET /dashboard/stats (DB-01's stat strip: study streak, weekly minutes, concepts mastered). Every other DB use case composes calls already documented in other §4 components instead of duplicating logic:

Recall AI	Version: 2.0
Software Architecture Document	Date: 14/Aug/26
RecallAI-SAD-v2.0	

			concepts-mastered over active plans only
DashboardStatsUtil	util	computeStreakDays, getVnWeekStartUtc, toVnDateKey (pure)	Timezone-correct (Asia/Ho_Chi_Minh) day/week boundaries; shared with DB-09 snooze
useAsyncResource	hook	data, loading, error, reload()	Per-panel data loader - independent loading/error state
MiniConceptGraph	component	reuses ConceptGraph + planApi (§4.5)	DB-02: embeds the real concept-graph visualization
TodayNudge / DeadlinePanel	component	call reviewQueueApi (SRE)	DB-04 reminder / DB-09 dismiss-snooze - Dashboard is the UI, SRE owns priority logic

Design notes

- No duplicated business logic. Every non-trivial computation Dashboard displays is owned by the component that already computes it elsewhere in §4; Dashboard only fetches and renders.
- Independent panel failure. Each data source is fetched independently, so one failing endpoint shows one block error instead of a full-page error (#169).
- VN-timezone boundary reused by DB-09. toVnDateKey/getVnWeekStartUtc are the single source for "what day is it in Vietnam" - DB-09's "Hoãn đến mai" must resolve to the same day boundary the streak counter uses.

Recall AI	Version: 2.0
Software Architecture Document	Date: 14/Aug/26
RecallAI-SAD-v2.0	

5. Deployment

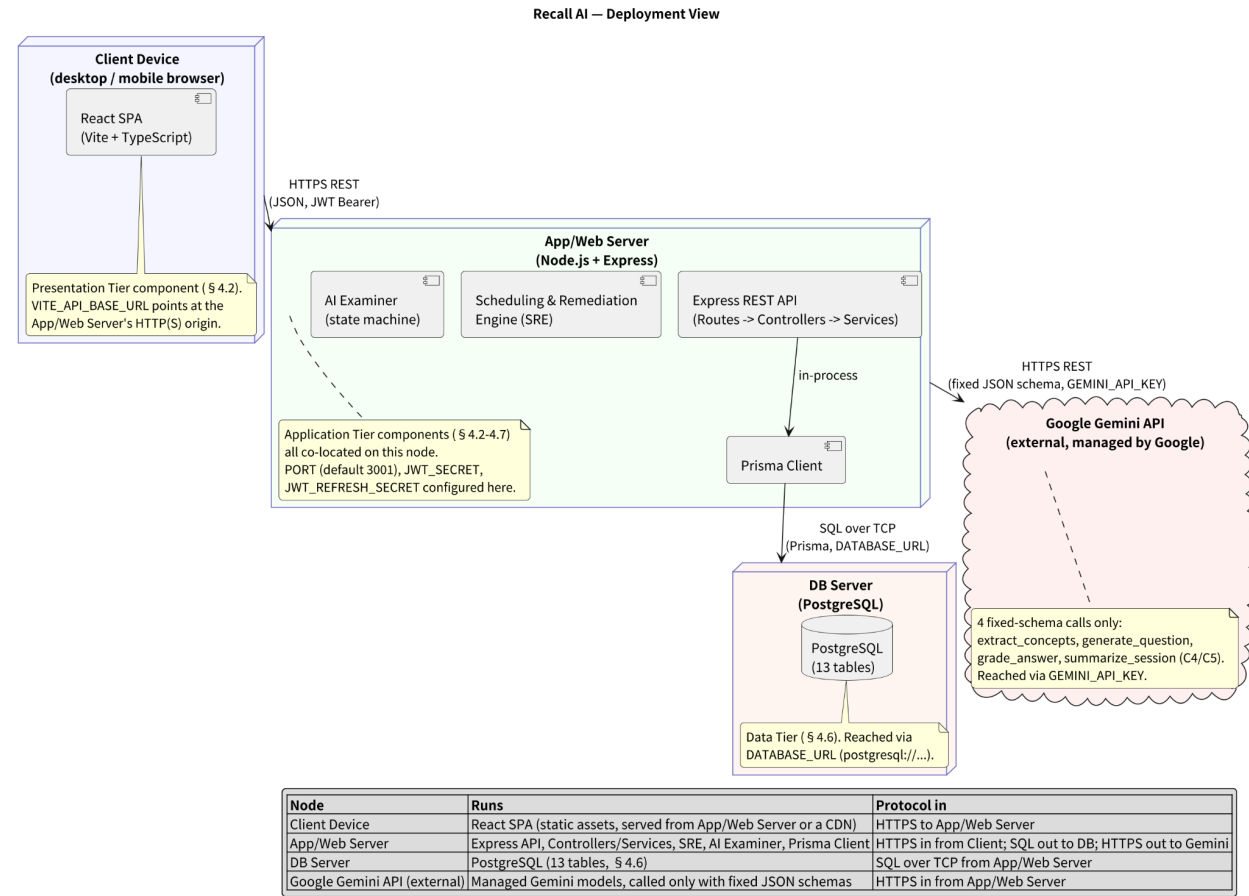


Figure 5.1 – Deployment View

5.1 Overview

Recall AI deploys as four physical/logical nodes: a client device running the browser SPA, an application/web server running the Express API, a database server running PostgreSQL, and the external Google Gemini API reached over the internet. This mirrors the 3-tier logical view already documented in §4 - the Deployment View maps those same components onto the machines that actually run them.

This repository has no Docker/IaC/hosting configuration, so the topology is the target deployment inferred directly from the application's own configuration (.env.example files, package.json dependencies) - not copied from an existing deployment manifest.

5.2 Node Descriptions and Protocols

NODE	RUNS	INBOUND / OUTBOUND PROTOCOL
Client Device (desktop or mobile browser)	The React SPA (Vite + TypeScript) - Presentation Tier.	Out: HTTPS REST, JSON, JWT Bearer auth, to the App/Web Server.

Recall AI	Version: 2.0
Software Architecture Document	Date: 14/Aug/26
RecallAI-SAD-v2.0	

App/Web Server (Node.js + Express)	Express REST API (Routes → Controllers → Services), Scheduling & Remediation Engine, AI Examiner state machine, Prisma Client.	In: HTTPS REST from Client. Out: SQL over TCP to DB Server (Prisma); HTTPS REST to Gemini API.
DB Server (PostgreSQL)	The persistence layer - 13 tables (§4.6).	In: SQL over TCP from App/Web Server only.
Google Gemini API (external, managed by Google)	Google's hosted Gemini models, called only through 4 fixed-schema calls per C4/C5.	In: HTTPS REST from App/Web Server only.

5.3 Notes

- The Client Device never talks to the DB Server or Gemini API directly - every request is mediated by the App/Web Server, keeping the AI-orchestration boundary (C4) and auth checks in one place.
- No CDN, load balancer, or container orchestration exists in the current codebase; this diagram documents the minimal topology the application requires to run

6. Implementation View

6.1 src/client - Presentation Tier

```

src/client/
├── public/           # Static assets served as-is (favicon, etc.)
├── src/
│   ├── assets/      # Bundled images/icons
│   ├── components/
│   │   ├── shared/  # Cross-feature composite components
│   │   └── ui/       # shadcn/ui primitives (buttons, dialogs, inputs,
... )
│   └── features/    # One folder per product feature, maps 1:1 to §4
components:
│   ├── auth/        # → §4.4 Authentication
│   ├── study-planner/ # → §4.5 Study-Plan & Concept-Graph
│   ├── interview/   # → §4.7 AI Examiner
│   ├── review-queue/ # → Scheduling & Remediation Engine
│   ├── focus/       # → §4.8 Focus Session
│   └── dashboard/   # → §4.9 Dashboard & Visualization
│   ├── hooks/       # Shared React hooks
│   └── lib/          # apiClient.ts, endpoints.ts, utils.ts - HTTP
client layer
├── pages/           # Route-level page components
├── types/           # Shared TS types/interfaces
└── utils/           # Framework-agnostic helper functions

```

Recall AI	Version: 2.0
Software Architecture Document	Date: 14/Aug/26
RecallAI-SAD-v2.0	

6.2 src/server - Application Tier

```

src/server/
├── prisma/
│   ├── schema.prisma      # §4.6 Database - single source of the 13-table model
│   └── migrations/
├── src/
│   ├── routes/             # Express routers - one file per resource
│   ├── controllers/        # HTTP boundary
│   ├── services/           # Business logic - maps to §4 components:
│   │   ├── auth.service.ts, user.service.ts      → §4.4 Authentication
│   │   ├── plan.service.ts, graph.service.ts     → §4.5 Study-Plan & Concept-Graph
│   │   ├── gemini.service.ts                    → AI Orchestration Boundary (C4)
│   │   ├── interview.service.ts, concept-result... → §4.7 AI Examiner
│   │   └── scheduling.service.ts, traceback.service.ts → Scheduling &
│   │       Remediation Engine
│   │   ├── focus-session.service.ts, session-note... → §4.8 Focus Session
│   │   └── dashboard.service.ts                    → §4.9 Dashboard
│   ├── schemas/           # Zod validation, incl. 4 fixed AI schemas
│   ├── middleware/        # auth, upload, error handling
│   ├── jobs/              # Background jobs
│   ├── config/            # Environment/config loading
│   ├── types/             # Shared TS types
│   ├── utils/             # Framework-agnostic helpers
│   └── __tests__/_        # Unit/integration tests

```

6.3 Notes

- The features/ → services/ naming is intentionally symmetric, making it straightforward to trace a §4 component end-to-end through both tiers.
- Build/runtime artifacts (dist/, node_modules/, uploads/) are intentionally omitted - only source-controlled main folders are listed.